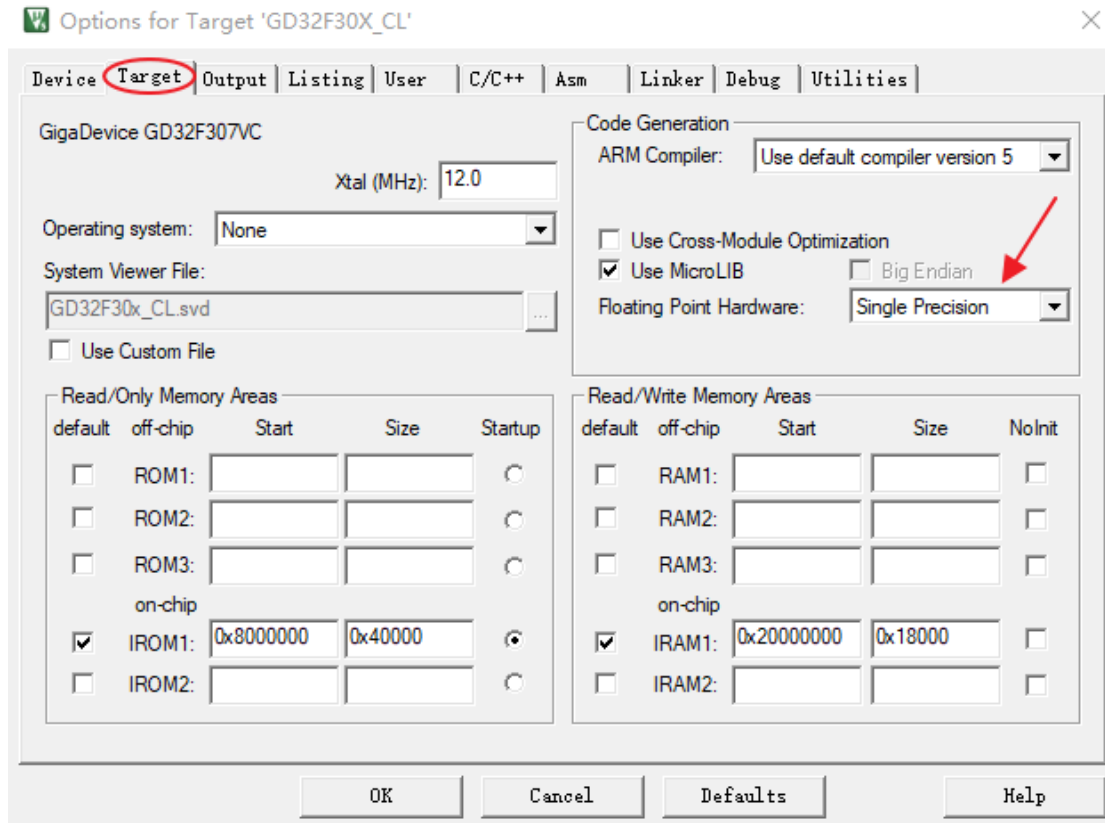


GD32 使用 FPU 以及 DSP 库的操作方法

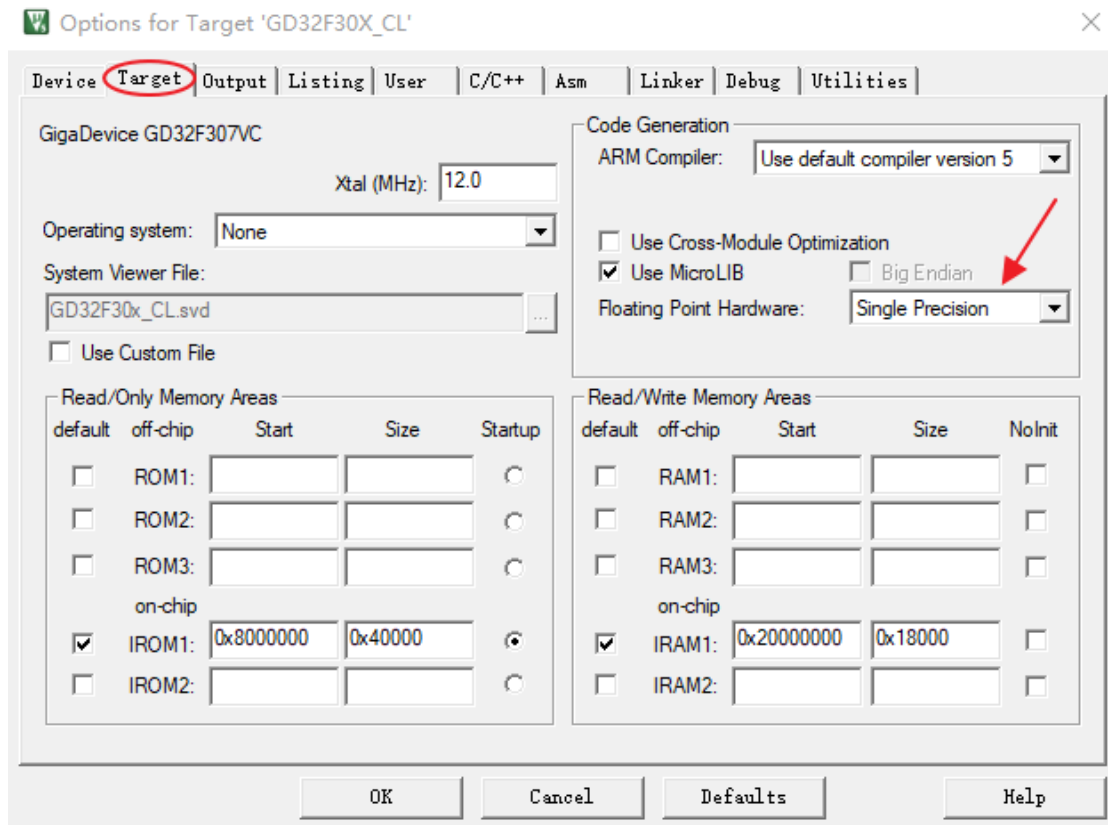
在 GD32 中的 F3 系列插件有两个版本，V1.0.1 的插件是不支持 FPU 的，V2.0.0 的版本才支持，如果安装的版本在 Target 中找不到下列的选项，请先安装新版本的插件。

在 F4 系列和 E103 系列插件都是支持 FPU 的。



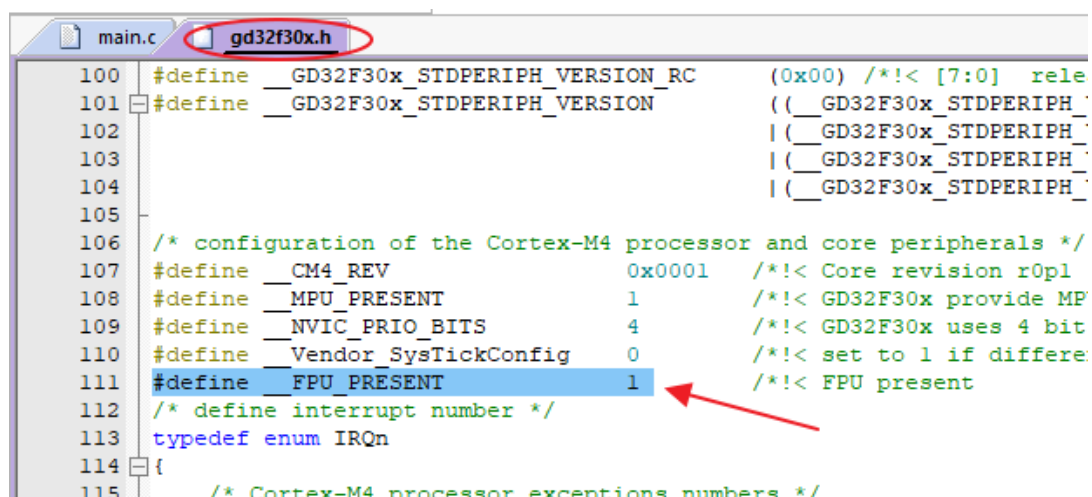
打开 FPU 的步骤

1. 在选项中找到上述的页面，选择使用 Single Precision。以下使用的是 KEIL5 编译器，其他版本的类似。

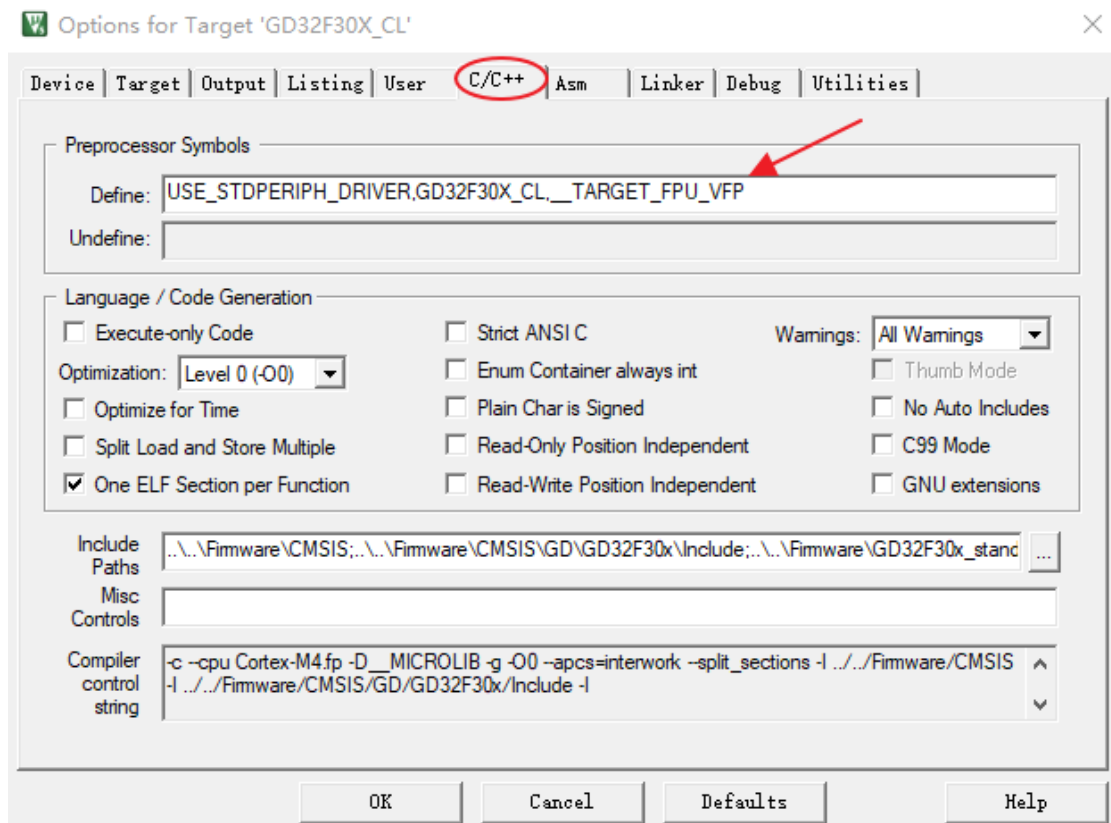


2. 以 F3 系列为例

在 gd32f3xx.h 中加入 `#define __FPU_PRESENT` 1



3. 在工程设置中添加预编译宏 `__TARGET_FPU_VFP`



4. 在 `system_gd32f3xx.c` 中的 `SystemInit()` 中加入

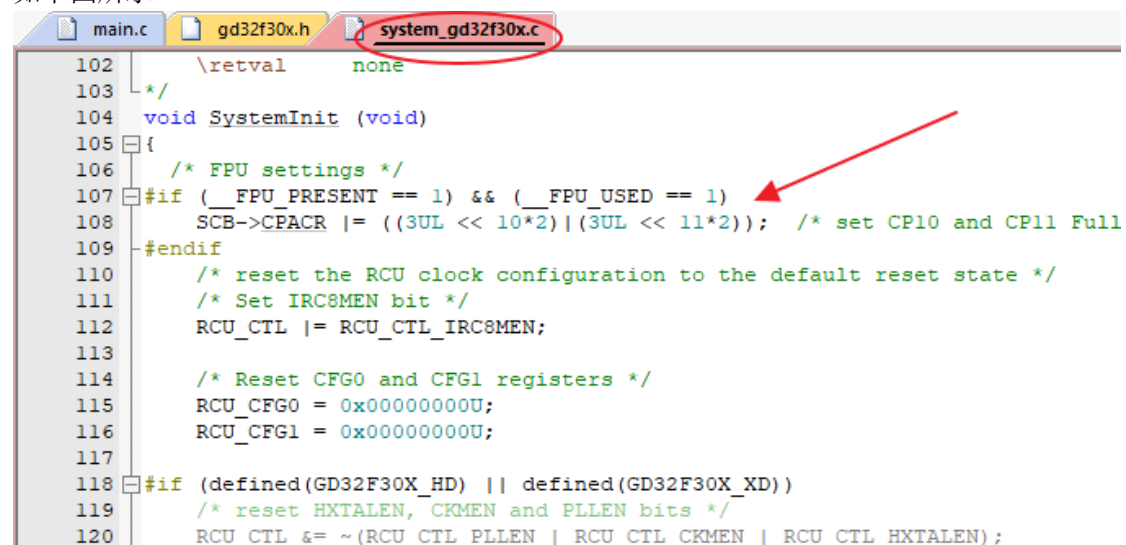
`/* FPU settings */ //如果库里没有这条语句，就需要加上`

`#if (__FPU_PRESENT == 1) && (__FPU_USED == 1)`

`SCB->CPACR |= ((3UL << 10*2) | (3UL << 11*2)); /* set CP10 and CP11 Full Access */`

`#endif`

如下图所示



使用 F4 系列和 E103 系列

1. 同样在 gd32f4xx.h 中加入 `#define __FPU_PRESENT` 1
2. 在工程设置中添加预编译宏 `__TARGET_FPU_VFP`
3. 在 system_gd32f4xx.c 中的 SystemInit() 中加入

```
/* FPU settings */ //如果库里没有这条语句，就需要加上
#if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
SCB->CPACR |= ((3UL << 10*2)|(3UL << 11*2)); /* set CP10 and CP11 Full Access */
#endif
```

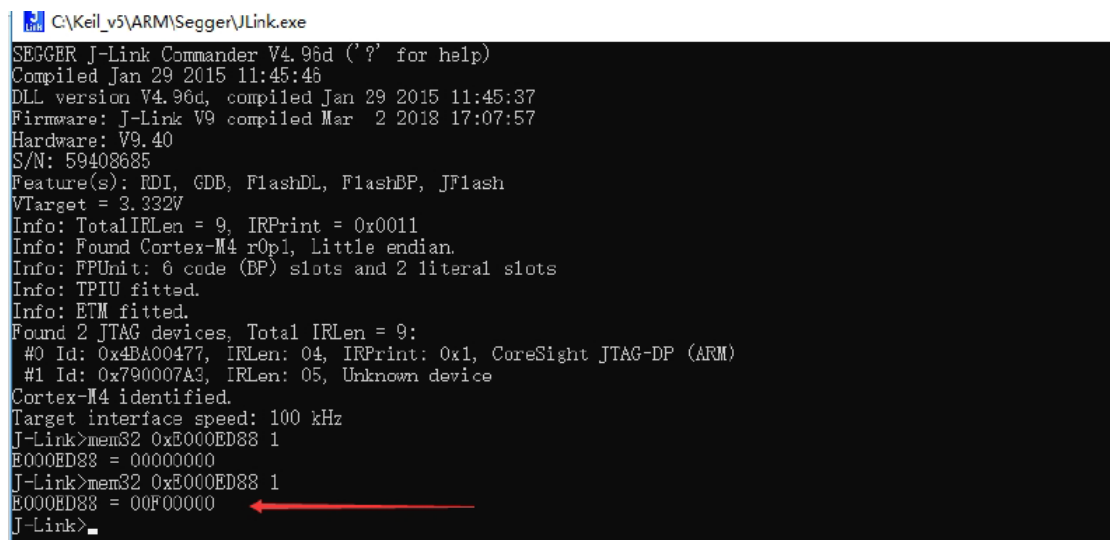
5. 重新编译下载烧录即可

检查 FPU 是否打开

1. 在 jlink 调试控制界面下，输入语句

```
mem32 0xE000ED88 1
```

具体看下图，得到的值为 00F0 0000，证明 FPU 已经开启



```
C:\Keil_v5\ARM\Segger\JLink.exe
SEGGER J-Link Commander V4.96d ('?' for help)
Compiled Jan 29 2015 11:45:46
DLL version V4.96d, compiled Jan 29 2015 11:45:37
Firmware: J-Link V9 compiled Mar 2 2018 17:07:57
Hardware: V9.40
S/N: 59408685
Feature(s): RDI, GDB, FlashDL, FlashBP, JFlash
VTarget = 3.332V
Info: TotalIRLen = 9, IRPrint = 0x0011
Info: Found Cortex-M4 r0p1, Little endian.
Info: FPUUnit: 6 code (BP) slots and 2 literal slots
Info: TPIU fitted.
Info: ETM fitted.
Found 2 JTAG devices, Total IRLen = 9:
#0 Id: 0x4BA00477, IRLen: 04, IRPrint: 0x1, CoreSight JTAG-DP (ARM)
#1 Id: 0x790007A3, IRLen: 05, Unknown device
Cortex-M4 identified.
Target interface speed: 100 kHz
J-Link>mem32 0xE000ED88 1
E000ED88 = 00000000
J-Link>mem32 0xE000ED88 1
E000ED88 = 00F00000
J-Link>
```

2. KEIL 调试浮点运算反汇编界面中会有 VLDR VADD 等浮点运算寄存器

如下图所示

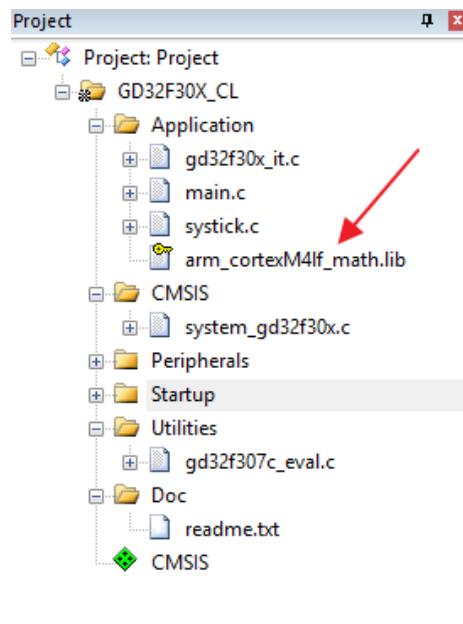
Disassembly				
0x080003CC	2002	MOVS	r0,#0x02	
0x080003CE	F7FFFF5F	BL.W	gd_eval_led_init (0x08000290)	
58:			gd_eval_led_init(LED5);	
0x080003D2	2003	MOVS	r0,#0x03	
0x080003D4	F7FFFF5C	BL.W	gd_eval_led_init (0x08000290)	
59:			*aq=a+b;	
0x080003D8	4820	LDR	r0,[pc,#128] ; @0x0800045C	
0x080003DA	ED900A00	VLDR	s0,[r0,#0x00]	
0x080003DE	4820	LDR	r0,[pc,#128] ; @0x08000460	
0x080003E0	EDD00A00	VLDR	s1,[r0,#0x00]	
0x080003E4	EE300A20	VADD.F32	s0,s0,s1	
0x080003E8	481E	LDR	r0,[pc,#120] ; @0x08000464	
0x080003EA	6800	LDR	r0,[r0,#0x00]	
0x080003EC	ED800A00	VSTR	s0,[r0,#0x00]	
60:			a=aq*b;	
61:			//a=arm_sqrt_f32(25.5,aq);	
62:				
0x080003F0	481C	LDR	r0,[pc,#112] ; @0x08000464	

没开启的情况如下图

```
Disassembly
57:      gd_eval_led_init(LED4);
0x08000540 2002      MOVS      r0,#0x02
0x08000542 F7FFFFF5 BL.W      gd_eval_led_init (0x08000404)
58:      gd_eval_led_init(LED5);
0x08000546 2003      MOVS      r0,#0x03
0x08000548 F7FFFFF5 BL.W      gd_eval_led_init (0x08000404)
59:      *aq=a+b;
0x0800054C 481D      LDR        r0,[pc,#116] ; @0x080005C4
0x0800054E 6801      LDR        r1,[r0,#0x00]
0x08000550 481D      LDR        r0,[pc,#116] ; @0x080005C8
0x08000552 6800      LDR        r0,[r0,#0x00]
0x08000554 F7FFFE18 BL.W      __aeabi_fadd (0x08000188)
0x08000558 491C      LDR        r1,[pc,#112] ; @0x080005CC
0x0800055A 6809      LDR        r1,[r1,#0x00]
0x0800055C 6008      STR        r0,[r1,#0x00]
60:      a=*aq*b;
61:      //a=arm_sqrt_f32(25.5,aq);
62:
```

使用 DSP 的步骤

1. 打开 FPU 之后，在工程中添加 ARM 官方浮点库 `"arm_cortexM4lf_math.lib"`
`"arm_math.h"`



2. 在需要使用浮点库函数的.c 文件中添加 `#include "arm_math.h"`（这个头文件中包含所有的函数接口）
3. 在工程设置中添加预编译宏 `ARM_MATH_CM4`

